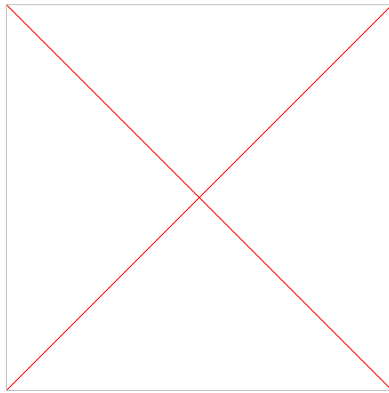




Database Systems: Functional Dependency

In this Lecture you will Learn about:

1. Prime & Non-Prime Attributes in RDBMS
2. Functional Dependency in RDBMS
3. Functional Dependency Types
4. Finding Super Keys and Candidate Keys

Prime or Key Attributes

Prime attributes are attributes that are part of any candidate key or the primary key of a relation.

These attributes are used to uniquely identify tuples (rows) in the relation.

In other words, prime attributes directly contribute to the identification of each tuple in the relation.

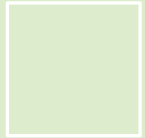
Non-prime attributes are attributes that are **not part of any candidate key or the primary key** of a relation.

These attributes are **not used for uniquely identifying** tuples in the relation but **provide additional information** about the entities represented by the relation.

Non-prime attributes are **dependent on the primary key or candidate keys** for their values.

Non Prime or Non Key Attributes

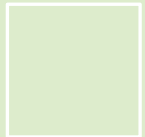
Functional Dependency



Functional dependencies (FDs) define the **relationship between attributes in a table.**



They specify how **one attribute or set of attributes can determine the value of another** attribute in the same table.



In other words, knowing the value of the first attribute(s) allows you to uniquely determine the value of the second attribute(s).

Functional Dependency



In a database, when information recorded in the same table uniquely determines other information stored in the same table.



It's critical to understand database dependencies since they serve as the foundation for database normalization.

Functional Dependency

Consider an **Employee** table with the following attributes:

- **Employee_ID**
- **Employee_Name**
- **Department**
- **Salary**

Functional Dependency Example:

- **Employee_ID** → **Employee_Name**

- The **Employee_ID** uniquely determines the **Employee_Name**.
- If you know the **Employee_ID**, you can find the corresponding **Employee_Name**.
- For instance, if **Employee_ID** = 101, you can be sure that the **Employee_Name** is "John Doe" (assuming unique **Employee_IDs**).
- Here, **Employee_ID** functionally determines **Employee_Name**.

Employee_ID	Employee_Name	Department	Salary
101	John Doe	HR	50000
102	Alice Smith	IT	60000

Functional Dependency

Consider a **Course_Enrollment** table with the following attributes:

- **Student_ID**
- **Course_ID**
- **Grade**

Functional Dependency Example:

- **Student_ID, Course_ID → Grade**

Student_ID	Course_ID	Grade
1	C101	A
1	C102	B
2	C101	C

- The combination of **Student_ID** and **Course_ID** uniquely determines the **Grade** of the student in that course.
- If you know the **Student_ID** and **Course_ID**, you can uniquely determine the **Grade**.
- For **Student_ID = 1** and **Course_ID = C101**, you can say that the Grade is A.
- This is a functional dependency where **Student_ID** and **Course_ID** together determine Grade.

Functional Dependency

- In a relational database management, functional dependency is a concept that specifies the relationship between two sets of attributes where one attribute determines the value of another attribute.
- It is denoted as $X \rightarrow Y$, where the attribute set on the left side of the arrow, **X** is called **Determinant**, and **Y** is called the **Dependent**.
- For example, if we have a table with attributes "A," "B," and "C," and attribute "A" determines the values of attributes "B" and "C," you would denote it as
- **A $\rightarrow$ B, C**
- This notation indicates that the value(s) in attribute "A" determines the value(s) in attributes "B" and "C."
- In other words, if you know the value of "A," you can determine the values of "B" and "C."

Functional Dependency

If the determinant X value is unique (different) then the dependent Y could have any value meaning:

- for same X , value of Y should be same.
- for different X, value of Y could be same or different.
- If a table contains an attribute as a primary key, then all other attributes in that table must be fully functionally dependent on the primary key.
- If $X \rightarrow Y$ holds true then $Y \rightarrow X$ may or may not hold.

Functional Dependency

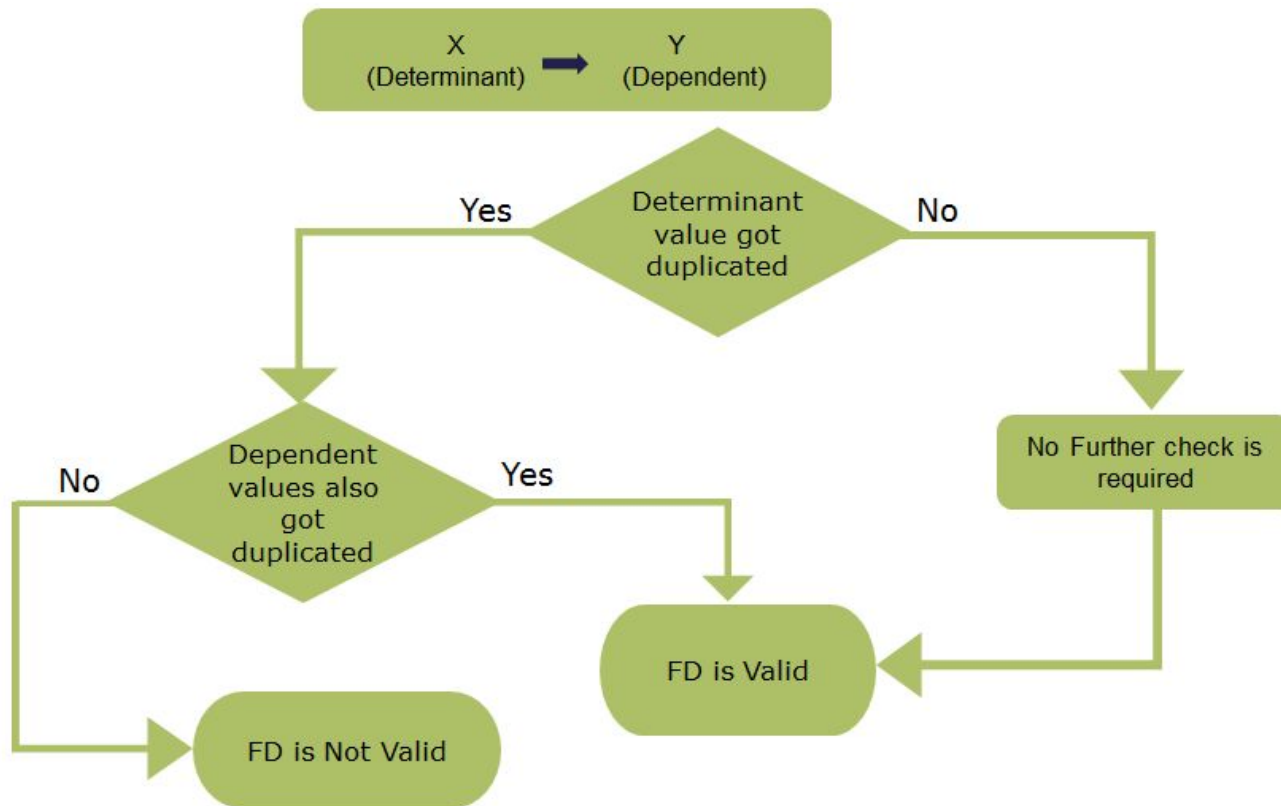
SIN ----> Name, Address, Birthdate

SIN, Course ----> DateCompleted

ISBN ----> Title

Functional Dependency

- Flow chart used for F.D validation check:



StudentID	StudentName
1	Ali
2	Sara
1	Ali

StudentID	StudentName
1	Ali
1	Ahmed

Trivial Functional Dependency

- In Trivial Functional Dependency, a dependent is always a subset of the determinant. i.e. If $X \rightarrow Y$ and Y is the subset of X , then it is called trivial functional dependency.
- Here, $\{\text{roll_no}, \text{name}\} \rightarrow \text{name}$ is a trivial functional dependency, since the dependent **name** is a subset of determinant set **{roll_no, name}**.

roll_no	name	age
42	abc	17
43	pqr	18
44	xyz	18

Non-Trivial Functional Dependency

- In **Non-trivial functional dependency**, the dependent is strictly not a subset of the determinant. i.e. If $X \rightarrow Y$ and **Y is not a subset of X**, then it is called Non-trivial functional dependency.
- Here, **roll_no** $\rightarrow$ **name** is a non-trivial functional dependency, since the dependent **name** is **not a subset of** determinant **roll_no**. Similarly, **{roll_no, name} $\rightarrow$ age** is also a non-trivial functional dependency, since **age** is **not a subset of {roll_no, name}**

roll_no	name	age
42	abc	17
43	pqr	18
44	xyz	18

Full Functional Dependency

- This concept applies when you have a composite key (a primary key made up of two or more columns).
- An attribute is fully functionally dependent if it relies on the entire composite key, and not just a piece of it.
- $(\text{Student_ID}, \text{Course_ID}) \rightarrow \text{Grade}$
- Grade depends on both attributes $\rightarrow$ Full Dependency

Partial Functional Dependency

- Occurs when:
 - A composite key exists
 - A non-key attribute depends on part of the key, not the whole key
- **Partial dependency** implies is a situation where a non-prime attribute is functionally dependent to a portion/part of a primary key/Candidate key.

Partial Functional Dependency

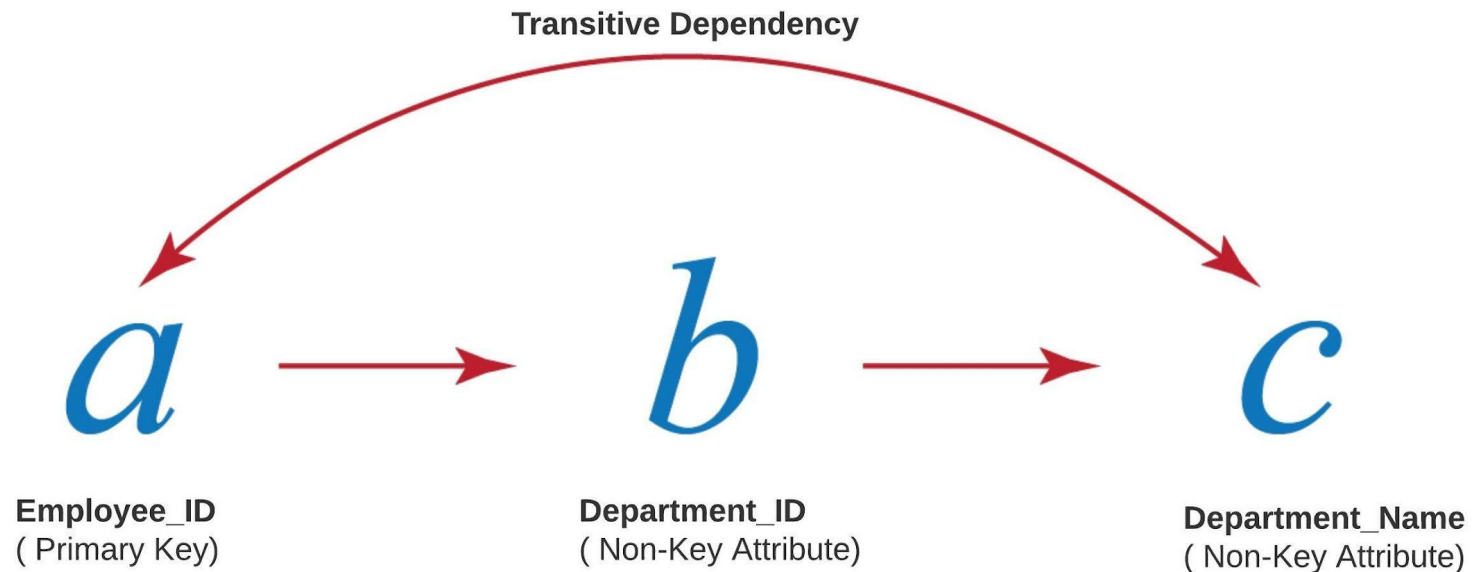
- Example

<StudentProject>

StudentID	ProjectNo	StudentName	ProjectName
S01	199	Katie	Geo Location
S02	120	Ollie	Cluster Exploration

- The prime key attributes are **StudentID** and **ProjectNo**.
- As stated, the non-prime attributes i.e. **StudentName** and **ProjectName** should be functionally dependent on part of a candidate key, to be Partial Dependent.
- The **StudentName** can be determined by **StudentID** that makes the relation Partial Dependent.
- The **ProjectName** can be determined by **ProjectID**, which that the relation Partial Dependent.

Transitive Functional Dependency



Transitive Functional Dependency

- When an indirect relationship causes functional dependency it is called Transitive Dependency.
- If $P \rightarrow Q$ and $Q \rightarrow R$ is true, then $P \rightarrow R$ is a transitive dependency.
- A transitive functional dependency in DBMS is a relational database table's relationship between attributes (columns).
- It occurs when one attribute's value determines another's value through an intermediary (a third) attribute.

Transitive Functional Dependency

- Consider a database table called "Student_Info" with the following attributes:
Student_ID (unique identifier for each student),
Student_Name,
Student_Address
Student_City.
- In this example, we can assume that Student_Address depends on Student_City, and Student_City depends on Student_ID.
- This creates a transitive dependency in DBMS, where Student_ID indirectly determines Student_Address.

Multi-Valued Functional Dependency

- In Multivalued functional dependency, entities of the dependent set are not dependent on each other. i.e. **If $a \rightarrow \{b, c\}$** and there exists no functional dependency between b and c, then it is called a multivalued functional dependency.
- Here, $\text{roll_no} \rightarrow \{\text{name}, \text{age}\}$ is a multivalued functional dependency, since the dependents name & age are not dependent on each other (i.e. $\text{name} \rightarrow \text{age}$ or $\text{age} \rightarrow \text{name}$ doesn't exist !)

roll_no	name	age
42	abc	17
43	pqr	18
44	xyz	18
45	abc	19

Armstrong's axioms/properties of functional dependencies

- Armstrong's axioms are a set of inference rules in relational databases.
- They help derive additional functional dependencies from a given set.
- William W. Armstrong formulated these axioms.
- They provide a formal basis for reasoning about dependencies in a schema.
- Useful for understanding relationships between attributes.
- Integral in the process of database normalization and optimization.

Armstrong's axioms/properties of functional dependencies

1. Reflexivity:

- **Axiom:** If $Y \subseteq X$, then $X \rightarrow Y$.
- **Explanation:** This states that an attribute or set of attributes always functionally determines itself. For example, if you have a set of attributes A, B , then $A, B \rightarrow A$ and $A, B \rightarrow B$.

2. Augmentation:

- **Axiom:** If $X \rightarrow Y$, then $XZ \rightarrow YZ$, where Z is any set of attributes.
- **Explanation:** If an attribute set X determines another set Y , adding additional attributes Z to both sides of the functional dependency does not affect the dependency. For example, if $\text{Employee_ID} \rightarrow \text{Employee_Name}$, then $\text{Employee_ID, Department} \rightarrow \text{Employee_Name, Department}$.

Armstrong's axioms/properties of functional dependencies

3. Transitivity:

- **Axiom:** If $X \rightarrow Y$ and $Y \rightarrow Z$, then $X \rightarrow Z$.
- **Explanation:** If an attribute set X determines Y , and Y determines Z , then X determines Z . For example, if $\text{Employee_ID} \rightarrow \text{Department}$ and $\text{Department} \rightarrow \text{Department_Location}$, then $\text{Employee_ID} \rightarrow \text{Department_Location}$.

4. Union:

- **Axiom:** If $X \rightarrow Y$ and $X \rightarrow Z$, then $X \rightarrow YZ$.
- **Explanation:** If an attribute set X determines both Y and Z , then X determines the combination of Y and Z .

Armstrong's axioms/properties of functional dependencies

5. Decomposition:

- Axiom: If $X \rightarrow YZ$, then $X \rightarrow Y$ and $X \rightarrow Z$.
- Explanation: If an attribute set X determines both Y and Z , then X determines Y and X determines Z separately.

6. Pseudo-Transitivity:

- Axiom: If $X \rightarrow Y$ and $YZ \rightarrow W$, then $XZ \rightarrow W$.
- Explanation: If X determines Y , and YZ determines W , then XZ will determine W .

Rules of Axioms

Use the Armstrong axioms to show that

1. $\text{isbn} \rightarrow \text{title, publisher}$
2. $\text{isbn, no} \rightarrow \text{author}$
3. $\text{publisher} \rightarrow \text{publisherURL}$

implies $\text{isbn} \rightarrow \text{publisherURL}$.

- | | |
|---|--|
| 4. $\text{title, publisher} \rightarrow \text{publisher}$ | <i>by reflexivity</i> |
| 5. $\text{isbn} \rightarrow \text{publisher}$ | <i>by transitivity using 1. and 4.</i> |
| 6. $\text{isbn} \rightarrow \text{publisherURL}$ | <i>by transitivity using 5. and 3.</i> |

Task

- Draw tables and illustrate FD such a full , partial, transitive, trivial and non trivial.
- Give examples of axioms and proof with example rules of axioms from given table.

Roll No	Name	Marks	Dept	Course
1	Ali	70	CS	C1
2	Sana	68	EE	C1
...	...	...	...	...